

DOCUMENTO DE DISEÑO

Code-Trekking - Proyecto de Título

Histórico de Revisiones

Versión	Fecha	Descripción/cambio	autor
1.0	17/08/2026	Inicio del documento	Ismael Horta - Danilo Gonzalez

Información del Proyecto

Organización	Duoc UC. Escuela de Informática y Telecomunicaciones
Sección	005-D
Proyecto (Nombre)	Code-Trekking
Fecha de Inicio	11/09/2026
Fecha de Término	26/11/2026
Patrocinador principal	code-trekking
Docente	Karla Roco Ramirez

Integrantes

Rut	Nombre	Correo
18.545.484-3	Ismael Horta	is.horta@duocuc.cl
18.947.774-0	Danilo Gonzalez	dana.gonzalezzc@duocuc.cl

1. Introducción

Este documento describe el diseño técnico de Code-Trekking, aplicación móvil orientada a personas interesadas en realizar trekking en la Región Metropolitana de Chile, que permite organizar salidas grupales mediante eventos con código de invitación, acceder a información centralizada de rutas, coordinarse a través de un foro por evento, registrar y exportar el recorrido mediante GPS, y disponer de un botón SOS que comparte la ubicación del usuario con sus contactos de emergencia.

El documento abarca la arquitectura de la solución (diagrama de arquitectura, de componentes y de comunicación entre servicios), el modelo de datos documental utilizado en Firebase, los diagramas de casos de uso, clases y secuencia de la funcionalidad principal, y el procedimiento de generación del ejecutable (APK) mediante GitHub Actions junto con las variables de entorno y los pasos para levantar el sistema.

Alcance Funcional del Sistema

El diseño descrito en este documento da soporte a las 78 historias de usuario del Product Backlog priorizado, organizadas en 12 épicas funcionales. La siguiente tabla resume el alcance, como contexto para los diagramas de arquitectura y de casos de uso.

#	Épica	Historias	Sprint
1	Autenticación, Sesión y Seguridad Base	HU-01 a HU-12	5 y 6
2	Cuenta, Términos y Perfil de Usuario	HU-13 a HU-22	7
3	Panel de Administración (Contenido Base)	HU-23 a HU-31	8
4	Gestión de Eventos de Trekking	HU-32 a HU-40	9
5	Mapa y Trazado de Rutas	HU-41 a HU-45	10
6	Registro de Ruta (Tracking GPS)	HU-46 a HU-50	11
7	Contactos de Emergencia y Alerta SOS	HU-51 a HU-57	12
8	Foro por Evento	HU-58 a HU-61	13
9	Clima y Ubicación	HU-62 a HU-65	13
10	Contenido Informativo	HU-66 a HU-70	14
11	Pagos y Suscripciones con PayPal	HU-71 a HU-76	15
12	Galería de Fotos	HU-77 a HU-78	14

2. Arquitectura

2.1 Stack Tecnológico

Code-Trekking está construido como una aplicación híbrida con Ionic y Angular, empaquetada mediante Capacitor para Android, y utiliza Firebase como backend. La siguiente tabla resume las principales tecnologías y librerías utilizadas, de acuerdo con las dependencias declaradas en package.json.

Capa	Tecnología	Uso en el proyecto
Frontend / UI	Angular 20, Ionic 8	Framework de la aplicación y componentes de interfaz (páginas, formularios, navegación por tabs).
Empaquetado nativo	Capacitor 8	Compila la app web como aplicación nativa Android y expone plugins nativos (cámara, geolocalización, filesystem, share, clipboard, haptics, status bar, keyboard, browser, app).
Geolocalización	@capacitor/geolocation, @capacitor-community/background-geolocation	Posición GPS en primer plano y en segundo plano, usada en el mapa, el tracking y la alerta SOS.
Mapas y rutas	leaflet, leaflet-routing-machine, leaflet-control-geocoder	Renderizado del mapa interactivo, trazado de rutas y geocodificación.
Backend	firebase, @angular/fire	Authentication, Firestore (base de datos documental) y Storage (archivos multimedia).
UI complementaria	swiper, ionicons, @ionic/pwa-elements	Carrusel de consejos, iconografía y elementos para funcionalidades PWA (captura de fotos en web).
Reactividad	RxJS	Manejo de observables para datos en tiempo real (foro, eventos, estado de sesión).
CI/CD	GitHub Actions	Pipeline de compilación y generación automática de la APK de Android.

2.2 Diagrama de Arquitectura

El diagrama de arquitectura representa la estructura general de la aplicación y muestra cómo se organizan sus capas, componentes y servicios para trabajar en conjunto. Incluye la interfaz de usuario, el control de acceso, la lógica de negocio, los modelos de datos y el almacenamiento mediante Firebase y localStorage. Además, muestra las reglas de seguridad y la integración con APIs externas para obtener información meteorológica, visualizar mapas y gestionar rutas.

Capa	Carpeta	Responsabilidad
Presentación	src/app/pages, src/app/components, src/app/tabs	Contiene las páginas de Ionic/Angular, los componentes reutilizables y la navegación por pestañas. Gestiona la interfaz de la aplicación y la interacción con el usuario.

Control de acceso	src/app/guards	Contiene los guards de navegación, que controlan el acceso a las rutas según el estado de la sesión y los permisos del usuario.
Lógica de negocio	src/app/core/services	Contiene los servicios que implementan y coordinan las funcionalidades de la aplicación, como seguridad, seguimiento de rutas y alertas SOS. También gestionan la comunicación con Firebase y los servicios externos.
Interceptores HTTP	src/app/core/interceptors	Gestiona las solicitudes HTTP que pasan por los interceptores, permitiendo aplicar controles comunes a las comunicaciones con servicios externos cuando corresponda.
Modelos de datos	src/app/core/models	Define las interfaces y estructuras TypeScript utilizadas para organizar y representar los datos de la aplicación.
Capa de datos	Firebase Authentication, Cloud Firestore y localStorage	Gestiona la autenticación de usuarios, el almacenamiento y la consulta de información en Firestore, y el almacenamiento local de datos en el dispositivo o navegador.
Seguridad de datos	Firestore Security Rules	Define las reglas de seguridad que controlan qué operaciones de lectura y escritura pueden realizar los usuarios sobre los datos almacenados en Firestore.
Servicios externos	APIs de OpenWeatherMap, OpenRouteService, OpenStreetMap, OpenTopoMap, Thunderforest, Wikiloc y OSRM	Proporciona funcionalidades externas relacionadas con información meteorológica, mapas, visualización del terreno, rutas y servicios de cálculo de recorridos. La comunicación se realiza

		mediante HTTPS/TLS cuando los servicios lo admiten.
--	--	---

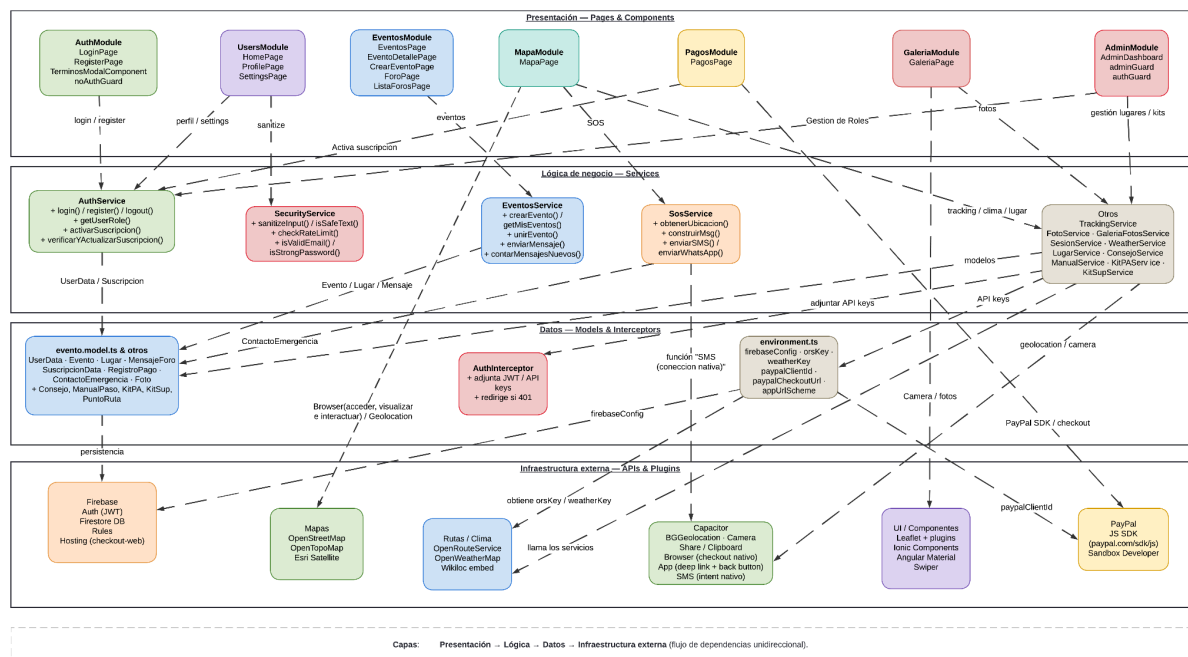


2.3 Diagrama de Componentes

Este diagrama detalla los componentes internos de la aplicación Angular/Ionic. Las siguientes tablas listan los componentes reales del repositorio, agrupados por tipo, como referencia para su elaboración.

Grupo	Componentes
Páginas de autenticación(pages/auth)	login, register
Páginas de usuario(pages/users)	home, profile, settings, eventos, crear-evento, evento-detalle, foro, lista-foros, mapa, galería, pagos
Páginas de administración(pages/admin)	dashboard
Shell de navegación	tabs
Componentes reutilizables	manual-supervivencia, términos-modal
Guards	auth.guard, no-auth.guard, admin.guard
Interceptores	auth.interceptor
Servicios del núcleo(core/services)	auth, session, security, sos, time, tracking, weather, weather-global, lugar, evento, concejo, manual, kit-primeros-ayudios, kit-supervivencia, galería-fotos, foto, tipo-cambio

Diagrama de componentes
Code-Trekking



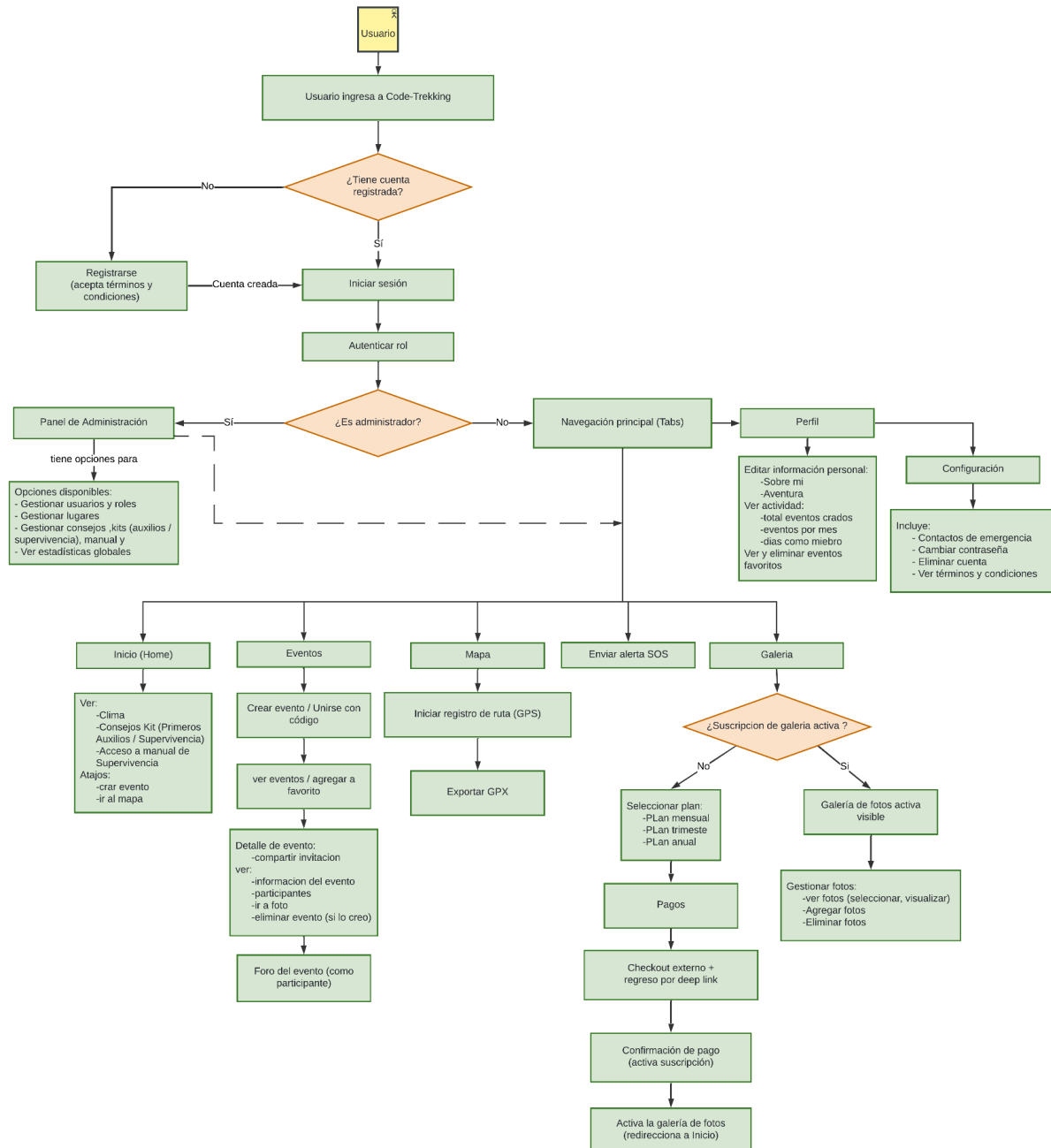
2.4 Diagrama de Comunicación entre Servicios

Este diagrama muestra el flujo de comunicación entre el cliente y los distintos servicios: las llamadas a Firebase Authentication para el registro e inicio de sesión, las lecturas y escrituras en tiempo real a Firestore (eventos, mensajes del foro, perfiles de usuario, contenido administrativo), la subida de archivos a Firebase Storage (fotos de perfil y galería), las consultas a las APIs externas de clima y enrutamiento, el flujo de pago con PayPal (SDK embebido en la versión web, y checkout externo más deep link codetrekking:// en la versión nativa), y el envío de la alerta SOS por SMS/WhatsApp utilizando la ubicación obtenida mediante los plugins de geolocalización de Capacitor.

Los principales puntos de integración externa, con el servicio/SDK utilizado y su propósito, son:

Servicio	Integración	Propósito
Firebase Authentication	@angular/fire/auth	Registro, inicio de sesión, recuperación de contraseña y manejo de sesión.
Firebase Firestore	@angular/fire/firestore	Base de datos documental: usuarios, eventos, foro, lugares, consejos, manual, kits, galería y pagos.
Firebase Storage	@angular/fire/storage	Almacenamiento de fotos de perfil y de la galería del usuario.
OpenWeatherMap	API REST (WEATHER_API_KEY)	Temperatura, descripción, humedad y viento según la ubicación del usuario.
OpenRouteService	API REST (ORS_KEY)	Cálculo de rutas e instrucciones paso a paso para el mapa.
Thunderforest	Teselas de mapa (THUNDERFOREST_KEY)	Capas de mapa de terreno y topográfico en Leaflet.
PayPal	PayPal JS SDK / Capacitor Browser	Procesamiento de pagos de suscripción, en la app (web) o vía checkout-web + deep link (nativo).
SMS / WhatsApp	Enlaces nativos (tel:, sms:, wa.me)	Envío del mensaje de alerta SOS a los contactos de emergencia.

Diagrama de comunicacion
Code-Trekking



2.5 Modelo de Datos (Modelo documental de Firebase)

Firestore es una base de datos NoSQL orientada a documentos: la información se organiza en colecciones de documentos en lugar de tablas relacionales, y las relaciones entre entidades se mantienen mediante identificadores de referencia (por ejemplo, el uid del usuario o el id del evento) almacenados como campos dentro de cada documento, en lugar de llaves foráneas. Las colecciones y sus campos corresponden exactamente a las interfaces TypeScript definidas en `core/models/evento.model.ts` y en los servicios del núcleo (`core/services`).

Colección	Interfaz	Campos principales
usuarios(id = uid)	UserData	uid, email, nombre, rol ('admin' 'user'), fotoBase64, estado, sobreMi, ocupacion, lugarSonado, mascotas, actividadesFavoritas, trekkingFavorito, proximoDesafio, preguntaSeguridad, respuestaSeguridad, contactosEmergencia (ContactoEmergencia[]), ultimoCambioNombre, estadisticas (EstadisticasUsuario), terminosAceptados, fechaAceptacionTerminos, versionTerminos, suscripcion (SuscripcionData).
usuarios/{uid}/pagos(subcolección)	RegistroPago	monto, plan ('mensual' 'trimestral' 'anual'), ordenId, fechaPago, uid.
eventos	Evento	id, nombre, descripcion, fecha, hora, lugarId, lugar (Lugar embebido), creadoPor ({uid, nombre}), codigoInvitacion (TRK-XXXXXX), privado, participantes (string[] de uids), participantesInfo (Participante[]), creadoEn, ultimoMensaje (previsualización del foro).
eventos/{id}/foro(subcolección)	MensajeForo	id, texto, autorUid, autorNombre, creadoEn.
lugares	Lugar	id, nombre, informacion, altitud, dificultad ('Baja'... 'Alta - Muy Alta'), distanciaKm, tiempoEstimadoHoras, equipamiento (string[]), DireccionPuntoInicio, latitud, longitud, requiereRegistroAcceso, requiereGuiaMontana, requierePagoEntrada, valorEntrada, requiereHorarioVisita, horarioVisita ({apertura, cierre}), requierePermiso, requiereMasInformacion, MasInformacion ({Texto, URL, Otro}), mapaRutaUrl.
consejos	Consejo	id, titulo, descripcion, creadoEn.
manual_supervivencia	ManualPaso	id, titulo, descripcion, icono, orden, creadoEn.
kit_primeros_auxilios	KitPrimerosAuxilios	id, nombre, descripcion, items (string[]), creadoPor, fechaCreacion.
kit_supervivencia	KitSupervivencia	id, nombre, descripcion, items (string[]), creadoPor, fechaCreacion.
galeriaFotos	FotoGaleria	id, uid, nombreUsuario, imageUrl, fechaSubida.

Ejemplo de documento de la colección eventos, tal como se almacena en Firestore:

```
{
  "nombre": "Travesía Cerro Provincia",
  "descripcion": "Salida de fin de semana, nivel intermedio",
}
```

```
{
  "fecha": "2026-10-18",
  "hora": "08:00",
  "lugarId": "abc123",
  "creadoPor": { "uid": "uid_danilo", "nombre": "Danilo González" },
  "codigoInvitacion": "TRK-7F3K9A",
  "privado": true,
  "participantes": ["uid_danilo", "uid_ismael"],
  "participantesInfo": [
    { "uid": "uid_danilo", "nombre": "Danilo González" },
    { "uid": "uid_ismael", "nombre": "Ismael Horta" }
  ],
  "creadoEn": "Timestamp"
}
```

Ejemplo de documento de la colección usuarios:

```
{
  "uid": "uid_ismael",
  "email": "is.horta@duocuc.cl",
  "nombre": "Ismael Horta",
  "rol": "user",
  "estado": "Con ganas de una travesía",
  "contactosEmergencia": [
    { "nombre": "Contacto 1", "telefono": "+56912345678" }
  ],
  "terminosAceptados": true,
  "fechaAceptacionTerminos": "2026-08-14",
  "versionTerminos": "1.0",
  "estadisticas": { "eventosCreados": 3, "eventosCreadosMes": 1, "ultimoMes": "2026-10" },
  "suscripcion": { "activa": false, "plan": null, "fechaInicio": "", "fechaVencimiento": "" }
}
```

2.6 Seguridad y Buenas Prácticas

La seguridad transversal de la aplicación se concentra en el servicio `security.service.ts` y en los `guards/interceptor` descritos en la sección 2.3, de acuerdo con las historias de usuario HU-05, HU-06, HU-09, HU-10, HU-11 y HU-12 del Product Backlog:

- Sanitización de entradas: todo texto ingresado por el usuario (nombres, descripciones, mensajes del foro, estado de ánimo) se sanitiza antes de almacenarse o mostrarse, para prevenir ataques de inyección de código (XSS).
- Límite de frecuencia (rate limiting): se restringe la frecuencia de acciones críticas (login, registro, creación de eventos, unión por código, envío de mensajes, cambio de nombre/contraseña, eliminación de cuenta) para mitigar el abuso automatizado.
- Guards de ruta: `authGuard` impide el acceso a rutas protegidas sin sesión activa, `noAuthGuard` impide acceder a login/registro con sesión activa, y `adminGuard` restringe el panel de administración solo a usuarios con rol admin.
- Interceptor HTTP: adjunta automáticamente el token de sesión a cada solicitud y redirige al login si el servidor responde con un error 401.
- Validación de datos críticos: se valida el formato y rango de datos sensibles (coordenadas GPS, identificadores de Firestore, teléfonos, correos y URLs externas) antes de procesarlos, incluyendo una whitelist de dominios externos permitidos.

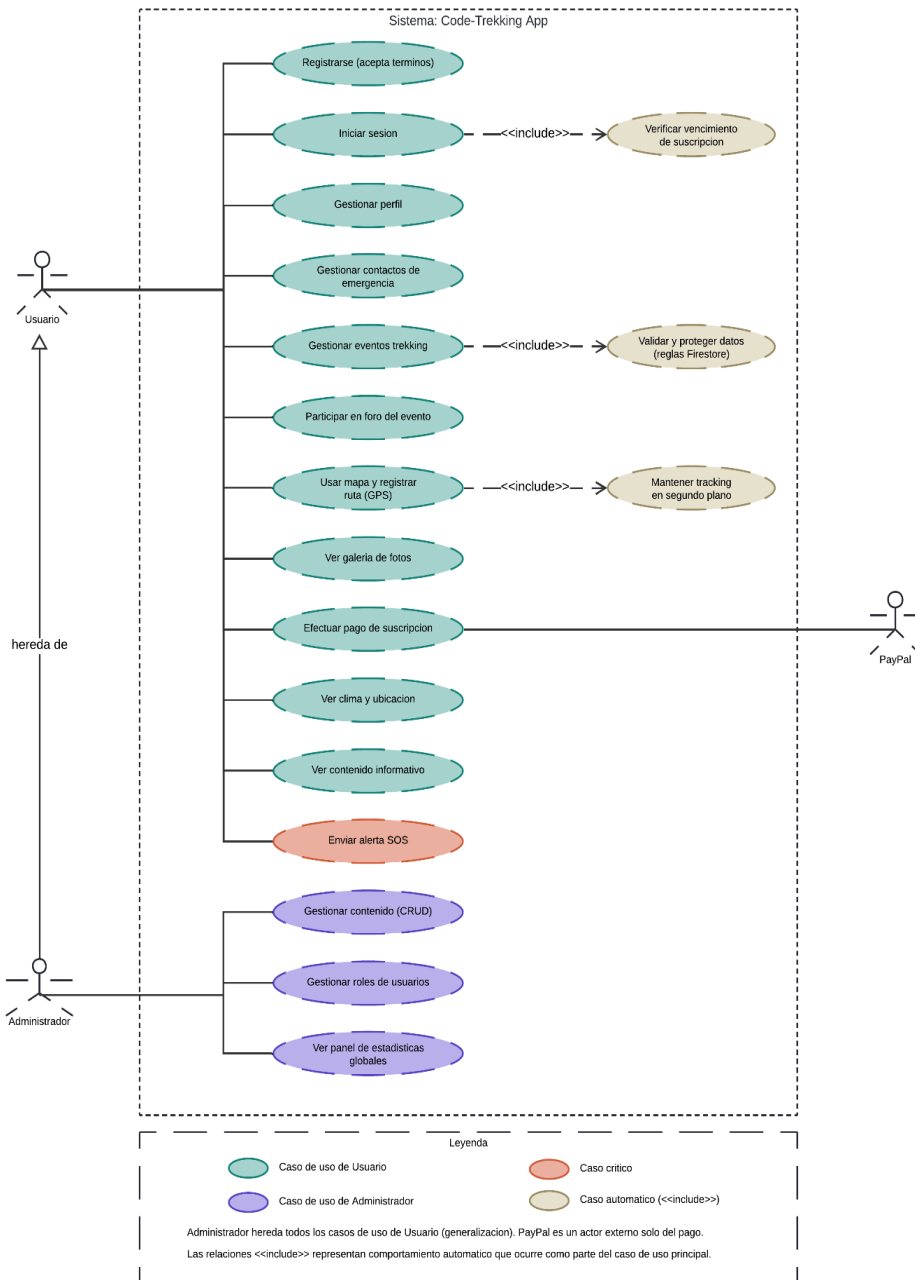
3. Diagramas

3.1 Casos de Uso

Representa las interacciones entre los actores del sistema (Usuario y Administrador) y las principales funcionalidades, agrupadas según las épicas del Product Backlog. La siguiente tabla resume los casos de uso por actor, como referencia para la elaboración del diagrama.

Actor	Épica	Casos de uso principales
Usuario	Autenticación y Cuenta	Registrarse, iniciar sesión, recuperar contraseña, cerrar sesión, cambiar/eliminar cuenta, aceptar y consultar términos y condiciones.
Usuario	Perfil de Usuario	Ver y editar perfil, cambiar foto y estado de ánimo, gestionar favoritos, ver estadísticas de actividad.
Usuario	Contactos de Emergencia y SOS	Gestionar contactos de emergencia, llamar/enviar SMS, activar la alerta SOS.
Usuario	Gestión de Eventos de Trekking	Crear evento, unirse por código, ver "Mis eventos", ver detalle, compartir código, abandonar/eliminar evento, marcar favorito.
Usuario	Foro por Evento	Enviar mensajes al foro, ver contador de mensajes nuevos; el organizador puede eliminar mensajes.
Usuario	Mapa y Tracking GPS	Ver mapa con capas, trazar ruta entre dos puntos, registrar y exportar el recorrido (GPX).
Usuario	Clima y Ubicación	Consultar el clima actual y el nombre de la ubicación.
Usuario	Contenido Informativo	Ver consejos, manual de supervivencia, kits de primeros auxilios y de supervivencia.
Usuario	Galería de Fotos	Ver, eliminar y visualizar fotos a pantalla completa.
Usuario	Pagos y Suscripciones	Seleccionar plan, pagar con PayPal (web o nativo), ver historial de pagos, cancelar el pago.
Administrador	Panel de Administración	Todas las mismas funciones que un usuario con rol user, más el acceso al panel, gestionar lugares, consejos, manual de supervivencia, kits y roles de usuario.

Vista de escenarios - Casos de uso Code-Trekking

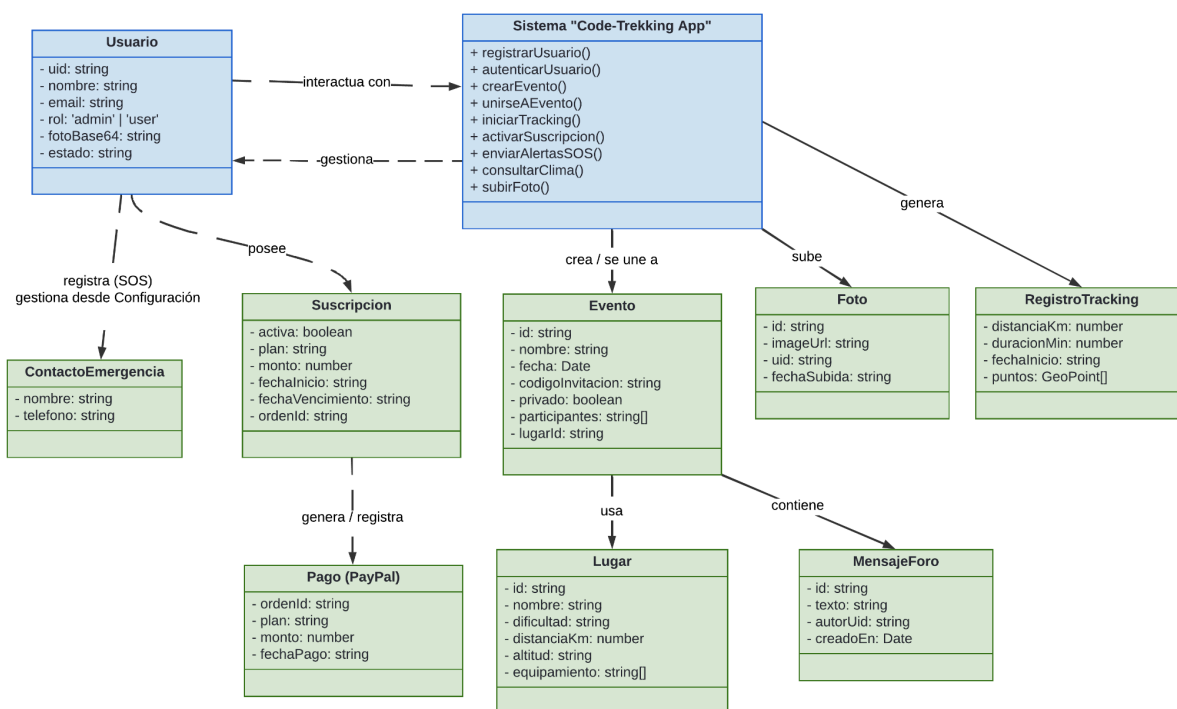


3.2 Clases (si corresponde)

Dado que el backend de Code-Trekking utiliza Firebase (modelo documental NoSQL) y no una base de datos relacional con un modelo de dominio orientado a objetos clásico, el diagrama de clases se limita a representar las principales interfaces de datos definidas en el código TypeScript de la aplicación (core/models/evento.model.ts y las interfaces declaradas en los servicios del núcleo) y su relación con los servicios de Angular que las consumen.

Interfaz	Definida en	Servicio Angular relacionado
Evento, Lugar, Participante, MensajeForo	core/models/evento.model.ts	evento.service.ts
Consejo	core/models/evento.model.ts	consejo.service.ts
ManualPaso	core/models/evento.model.ts	manual.service.ts
KitPrimerosAuxilios	core/models/evento.model.ts	kit-primeros-auxilios.service.ts
KitSupervivencia	core/models/evento.model.ts	kit-supervivencia.service.ts
UserData, ContactoEmergencia, EstadísticasUsuario	auth.service.ts	auth.service.ts, session.service.ts
SuscripcionData, RegistroPago	auth.service.ts	auth.service.ts (pagos y suscripción)
FotoGaleria	galeria-fotos.service.ts	galeria-fotos.service.ts, foto.service.ts

Diagrama de clases Code-Trekking



Privados (-): todos los atributos de los modelos son privados; el acceso se realiza a través de los servicios de Angular (AuthService, EventoService, etc.).

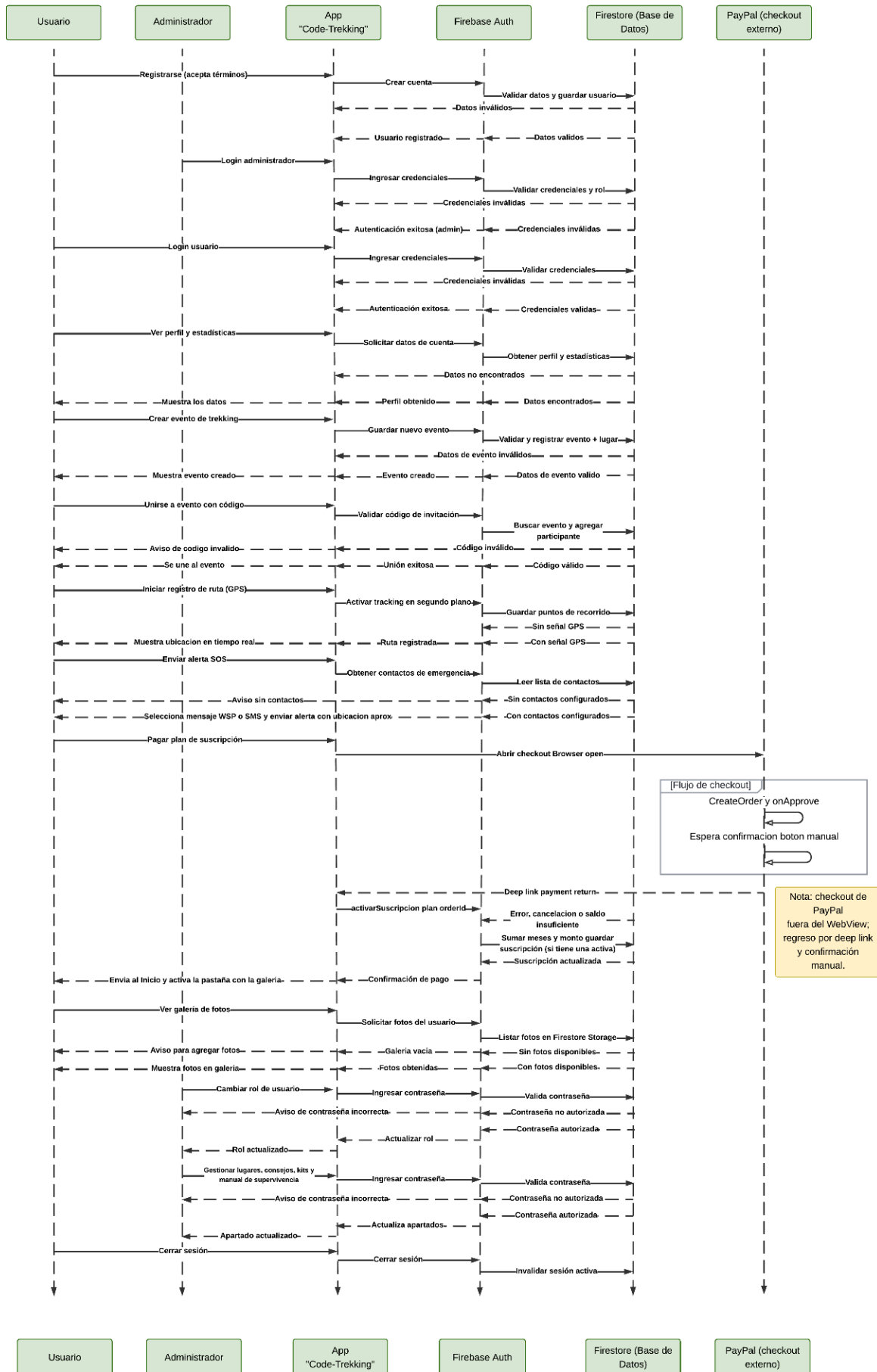
3.3 Secuencia de la Funcionalidad Principal

Se documenta como funcionalidad principal el flujo de activación de la alerta SOS (HU-55, HU-56 y HU-57), por corresponder al componente de seguridad central del proyecto, de acuerdo con lo descrito en el Caso de Proyecto de Título. El diagrama de secuencia muestra la interacción entre el Usuario, la aplicación, el plugin de geolocalización de Capacitor (Geolocation o Background

Geolocation según la plataforma), Firestore y los canales de salida (SMS y WhatsApp). Los pasos principales del flujo son:

- El Usuario presiona el botón SOS en la aplicación.
- La aplicación consulta en Firestore (colección usuarios) si el usuario tiene contactos de emergencia registrados.
- Si no tiene contactos registrados, el sistema bloquea la activación y redirige al usuario a Configuración (HU-56).
- Si tiene contactos registrados, la aplicación solicita la ubicación actual: en dispositivos nativos mediante Background Geolocation y en la versión web mediante la Geolocation API del navegador (HU-57).
- La aplicación consulta en Firestore si el usuario participa en un evento en curso, para incluirlo en el mensaje.
- La aplicación construye el mensaje de emergencia con la ubicación, la hora, el evento activo (si corresponde) y un enlace de Google Maps.
- La aplicación envía el mensaje por SMS y/o WhatsApp a cada uno de los contactos de emergencia registrados.
- La aplicación confirma al Usuario que la alerta fue enviada.

Diagrama de secuencia Code-Trekking



4. Generación del Ejecutable (APK) y Puesta en Marcha del Sistema

A diferencia de un despliegue basado en contenedores Docker, Code-Trekking es una aplicación móvil híbrida donde la lógica de datos se apoya en Firebase, por lo que la generación del ejecutable se automatiza mediante un pipeline de GitHub Actions que compila y empaqueta la APK de Android.

4.1 APK generada en GitHub Actions

El workflow `.github/workflows/build-apk.yml` se ejecuta de forma manual (`workflow_dispatch`) o automáticamente con cada commit a la rama `main`.

La APK se obtiene ingresando a la pestaña Actions del repositorio en GitHub, seleccionando la ejecución del workflow "Build Android APK" y descargando el artefacto `app-debug-apk`, que se descomprime para obtener `app-debug.apk` e instalarse en un dispositivo Android habilitando la instalación de aplicaciones de origen desconocido. Corresponde a una APK de depuración (`assembleDebug`), pensada para pruebas internas.

4.2 Variables de Entorno

El proyecto centraliza sus credenciales y claves de servicios externos en un archivo `.env`, excluido del repositorio mediante `.gitignore`. Al ejecutar `npm run config` (invocado automáticamente por los scripts `start`, `build` y `watch`), el script `scripts/set-env.ts` genera `src/environments/environment.ts` y `environment.prod.ts` a partir de esas variables. En GitHub Actions, el mismo contenido se entrega mediante el secreto del repositorio `ENV_FILE`.

Variable	Descripción
<code>FIREBASE_API_KEY</code>	API Key del proyecto Firebase.
<code>FIREBASE_AUTH_DOMAIN</code>	Dominio de autenticación de Firebase Authentication.
<code>FIREBASE_PROJECT_ID</code>	Identificador del proyecto de Firebase.
<code>FIREBASE_STORAGE_BUCKET</code>	Bucket de Firebase Storage (fotos de perfil, galería).
<code>FIREBASE_MESSAGING_SENDER_ID</code>	Identificador del remitente de Firebase Cloud Messaging.
<code>FIREBASE_APP_ID</code>	Identificador de la app dentro del proyecto Firebase.
<code>WEATHER_API_KEY</code>	Clave del servicio meteorológico externo (información climática).
<code>ORS_KEY</code>	Clave de OpenRouteService, usada para el trazado de rutas en el mapa.
<code>THUNDERFOREST_KEY</code>	Clave de Thunderforest, usada para las capas de mapa (terreno/topográfico).
<code>PAYPAL_CLIENT_ID</code>	Client ID de la cuenta de comercio de PayPal, usado por el SDK de pagos.
<code>PAYPAL_CURRENCY</code>	Moneda del cobro en PayPal (por defecto USD).
<code>PAYPAL_CHECKOUT_URL</code>	URL del checkout-web publicado en Firebase Hosting, usado en el pago nativo.
<code>APP_URL_SCHEME</code>	Esquema de deep link de la app (por defecto <code>codetrekking</code>), usado para el retorno desde PayPal en Android/iOS.

4.3 Cómo Levantar el Sistema

Los siguientes pasos corresponden a los indicados en el README.md del repositorio:

https://github.com/IsmaelH3030/Documentacion_ProyectoAPT_Code-Trekking.git, para ejecutar el proyecto Code-Trekking de forma local, es necesario contar con Node.js, npm, Angular CLI e Ionic CLI instalados, además de una copia del repositorio y las credenciales requeridas por los servicios externos utilizados por la aplicación.

4.3.1 Clonar el repositorio

Ejecutar los siguientes comandos en una terminal para descargar el proyecto e ingresar a su carpeta:

- `git clone https://github.com/DaniloGCh/code-trekking.git`
- `cd code-trekking`

4.3.2 Instalar las dependencias

Instalar las herramientas de Angular e Ionic, junto con las dependencias necesarias para ejecutar el proyecto:

- `npm install -g @angular/cli @ionic/cli`
- `npm install`

4.3.3 Configurar las variables de entorno

Por motivos de seguridad, las credenciales no se encuentran publicadas en el repositorio. Para configurar los servicios utilizados por la aplicación, se debe solicitar al administrador o dueño del proyecto el contenido del archivo `.env`.

Luego, crear el archivo `.env` en la raíz del proyecto, al mismo nivel que `package.json`, pegar el contenido proporcionado y guardar los cambios.

4.3.4 Preparar y ejecutar la aplicación

Ejecutar los siguientes comandos en la terminal para instalar las dependencias adicionales, compilar el proyecto e iniciar el servidor de desarrollo:

- `npm install dotenv`
- `npm install @ionic/pwa-elements`
- `npm run build`
- `ionic serve`

Como alternativa, se puede iniciar el servidor utilizando `npm start` o `ng serve`, según la configuración del proyecto.

Una vez iniciado el servidor, abrir la dirección local que indique la terminal para acceder a la aplicación desde el navegador.

4.3.5 Descargar la aplicación en formato APK

Si se requiere instalar la aplicación en un dispositivo Android, se puede acceder al repositorio en GitHub, ingresar a la sección **Actions** y revisar la ejecución correspondiente al último commit de la rama main. Si el proceso de compilación finalizó correctamente y el flujo publica el archivo como artefacto, se podrá descargar el APK generado.

4.4 Estructura de Carpetas del Proyecto

La siguiente estructura resumen la organización del código fuente dentro de src/app, de utilidad para ubicar cada componente descrito en este documento:

```
src/app/  
├── pages/  
│   ├── auth/      (login, register)  
│   ├── users/     (home, profile, settings, eventos,  
│                   crear-evento, evento-detalle, foro,  
│                   lista-foros, mapa, galeria, pagos)  
│   ├── admin/     (dashboard)  
│   └── tabs/      (tab1, tab2, tab3)  
├── components/    (manual-supervivencia, terminos-modal)  
├── guards/        (auth.guard, no-auth.guard, admin.guard)  
├── core/  
│   ├── services/  (auth, session, security, sos, time, tracking, weather,  
│                   weather-global, lugar, evento, consejo, manual,  
│                   kit-primeros-ayudios, kit-supervivencia,  
│                   galeria-fotos, foto, tipo-cambio)  
│   ├── models/    (evento.model.ts)  
│   └── interceptors/ (auth.interceptor)
```

5. Conclusión

El presente diseño refleja la arquitectura híbrida y adoptada para Code-Trekking (Ionic, Angular, Capacitor y Firebase), el modelo de datos documental utilizado en Firestore extraído directamente de las interfaces definidas en el código fuente del proyecto-, los principales flujos de interacción del sistema, y el procedimiento de generación del ejecutable y puesta en marcha mediante GitHub Actions. Este diseño es consistente con el Product Backlog priorizado, el Sprint Backlog, el Sprint Planning y el Manual Técnico de Despliegue ya elaborados para el proyecto, y sirve de referencia técnica para el desarrollo e incorporación de los diagramas visuales (arquitectura, componentes, comunicación entre servicios, casos de uso, clases y secuencia) señalados en los espacios reservados de este documento.